

# Paper

“latex

## 1 Summary

Bài báo *Mixture-of-Experts Knowledge Graph Retrieval-Augmented Generation for Multi-Agent LLM-based Recommendation* nghiên cứu bài toán gợi ý bằng LLM có hỗ trợ *Knowledge Graph Retrieval-Augmented Generation* (KG-RAG). Bối cảnh của bài báo là các LLM ngày càng được dùng trong recommender system vì có khả năng hiểu ngôn ngữ, diễn giải user intent và khai thác semantic item tốt. Tuy nhiên, nếu chỉ dựa vào parametric knowledge bên trong LLM, hệ thống dễ gặp các vấn đề như hallucination, thiếu tri thức domain mới, hoặc không biết các item mới chưa có trong dữ liệu pre-training.

Để khắc phục vấn đề này, các hệ KG-RAG recommender truy xuất tri thức từ knowledge graph rồi đưa vào prompt của LLM. Knowledge graph có lợi vì nó chứa tri thức có cấu trúc, có thể cập nhật, và mô tả quan hệ giữa item, entity, relation, genre, actor, director, artist hoặc thuộc tính khác. Tuy nhiên, paper cho rằng các phương pháp KG-RAG hiện tại vẫn có ba hạn chế lớn.

Hạn chế thứ nhất là *fixed retrieval granularity*. Các query của user có độ khó khác nhau. Một query đơn giản có thể không cần truy xuất KG hoặc chỉ cần vài triple trực tiếp. Ngược lại, một query phức tạp có thể cần multi-hop subgraph hoặc connected graph để suy luận. Nếu luôn dùng retrieval sâu cho mọi query, hệ thống bị *over-retrieval*, đưa nhiều tri thức dư thừa vào prompt, gây nhiễu và tăng chi phí. Nếu luôn dùng retrieval nông, hệ thống bị *under-retrieval*, thiếu tri thức cần thiết cho query phức tạp. Hình 1 của paper minh họa rõ vấn đề này: existing RAGRec dùng cùng một granularity cho easy, medium và hard query, còn MixRAGRec chọn granularity phù hợp hơn theo từng query.

Hạn chế thứ hai là *structure-to-text gap*. Tri thức từ KG là graph-structured data, trong khi LLM nhận input dạng text tuyến tính. Nếu chỉ serialize triple hoặc subgraph thành text thô, hệ thống có thể làm mất cấu trúc graph, đưa vào nhiều relation không liên quan, hoặc làm tri thức trở nên khó dùng đối với LLM. Vì vậy, cần một bước alignment để biến tri thức KG thành natural-language snippet vừa ngắn gọn vừa hữu ích cho recommendation.

Hạn chế thứ ba là retrieval granularity không có nhãn giám sát trực tiếp. Không có ground-truth nói rằng query này phải dùng no retrieval, triple retrieval hay subgraph

retrieval. Chất lượng của lựa chọn retrieval chỉ được phản ánh gián tiếp qua recommendation cuối cùng. Vì retrieval, alignment và recommendation phụ thuộc lẫn nhau, nếu tối ưu end-to-end đơn giản thì model có thể học một preference cố định cho một loại expert thay vì chọn expert theo query.

Để giải quyết các vấn đề trên, bài báo đề xuất *MixRAGRec*, một framework KG-RAG multi-agent gồm ba agent chính: *Mixture-of-Experts Retrieval Agent*, *Knowledge Preference Alignment Agent*, và *Contrastive Learning-reinforced Recommendation Agent*. Ba agent này được tối ưu bằng một framework chung tên là *Mixture-of-Experts Multi-Agent Policy Optimization* (MMAPO).

Bài toán được định nghĩa như sau. Với mỗi instance, hệ thống nhận một user query  $q$  mô tả intent và context của user, ví dụ lịch sử tương tác. Hệ thống cần chọn item  $o$  từ candidate set  $\mathcal{O}$ . Ngoài ra, hệ thống có một KG:

$$G = (\mathcal{V}, \mathcal{R}, \mathcal{T})$$

trong đó  $\mathcal{V}$  là tập entity,  $\mathcal{R}$  là tập relation, và:

$$\mathcal{T} \subseteq \mathcal{V} \times \mathcal{R} \times \mathcal{V}$$

là tập triples. Với mỗi query  $q$ , retrieval agent chọn một expert  $e$  từ tập:

$$\mathcal{E}_{exp} = \{e_1, e_2, e_3, e_4\}$$

mỗi expert trả về knowledge  $K_e$  ở một mức granularity khác nhau.

Objective tổng quát của MixRAGRec là:

$$(\theta^*, \psi^*, \omega^*) = \arg \max_{\theta, \psi, \omega} \mathbb{E}_{(q_i, \mathcal{O}_i, o_i^*) \sim \mathcal{D}} \left[ \mathbb{E}_{e_i \sim \pi_\theta(\cdot | s_i)} \mathbb{E}_{\tilde{K}_i \sim \pi_\psi(\cdot | q_i, K_{i, e_i})} \mathbb{E}_{o_i \sim \pi_\omega(\cdot | q_i, \tilde{K}_i, \mathcal{O}_i)} R(q_i, \mathcal{O}_i, o_i^*; e_i, \tilde{K}_i, o_i) \right]$$

Trong đó  $\pi_\theta$  là policy của retrieval agent,  $\pi_\psi$  là policy của alignment agent,  $\pi_\omega$  là policy của recommendation agent, còn  $o_i^*$  là ground-truth item.

Thành phần đầu tiên là *Mixture-of-Experts Retrieval Agent*. Agent này chọn một trong bốn retrieval experts. Expert 1 là *DirectGenerator*, không truy xuất KG:

$$K_1 = \emptyset$$

Expert này phù hợp với query đơn giản hoặc query mà LLM đã đủ tri thức để trả lời.

Expert 2 là *TripleRetriever*. Expert này truy xuất các KG triples liên quan nhất đến query bằng semantic similarity. Đầu tiên, entity và triple trong KG được encode bằng một pretrained language encoder. Với query:

$$z_q = \text{Encode}(q) \in \mathbb{R}^d$$

Với triple  $(h, r, t)$ :

$$z_{(h,r,t)} = \text{Encode}(\text{Concat}(x_h, x_r, x_t)) \in \mathbb{R}^d$$

Sau đó, expert lấy top- $M$  triples:

$$K_2 = \text{TopM}_{(h,r,t) \in \mathcal{T}} \text{sim}(z_q, z_{(h,r,t)})$$

Expert này phù hợp với các query cần một số fact trực tiếp, ví dụ quan hệ actor, director, genre hoặc thuộc tính item.

Expert 3 là *SubgraphRetriever*. Expert này trước hết chọn seed entities liên quan nhất:

$$S = \text{TopM}_{n \in \mathcal{V}} \text{sim}(z_q, z_n)$$

Sau đó mở rộng mỗi seed thành  $k$ -hop neighborhood subgraph:

$$K_3 = \bigcup_{n \in S} G_n^{(k)}$$

Expert này phù hợp với query cần multi-hop context hoặc quan hệ rộng hơn giữa nhiều entity.

Expert 4 là *ConnectedGraphRetriever*. Expert này dùng cho query phức tạp cần một graph liên thông và coherent hơn. Trước hết, expert lấy seed set  $S$ , sau đó chạy *Personalized PageRank* từ seed để lấy các entity quan trọng  $V_{PPR}$ . Trên induced graph  $G[V_{PPR}]$ , expert gán cost cho mỗi edge dựa trên độ phù hợp giữa query và relation type:

$$\text{cost}(e) = 1 - \cos(z_q, z_{r(e)})$$

Sau đó dùng *Kruskal's algorithm* để tìm minimum-cost spanning tree:

$$K_4 = \text{MST}(G[V_{PPR}])$$

Cách này giúp tạo một connected subgraph chứa các entity quan trọng và các relation phù hợp nhất với query.

Thành phần thứ hai là *Knowledge Preference Alignment Agent*. Vì output của retrieval expert là structured KG knowledge, agent này chuyển nó thành natural-language snippet  $\tilde{K}$  phù hợp hơn với LLM. Đầu tiên, hệ thống dùng một template module:

$$\text{Temp}(K)$$

để verbalize triples hoặc graph edges. Với triple  $(h, r, t)$ , template có thể tạo câu dạng “{head} has {relation} {tail}”. Với subgraph hoặc connected graph, module chọn các relational statements quan trọng, ví dụ các edges trên short paths giữa retrieved entities.

Sau đó alignment agent refine draft này:

$$\tilde{K} = \text{Align}_\psi(q, \text{Temp}(K))$$

Mục tiêu là tạo ra đoạn knowledge ngắn gọn, giảm nhiễu, giữ thông tin liên quan đến preference và dễ dùng cho recommendation.

Thành phần thứ ba là *Contrastive Learning-reinforced Recommendation Agent*. Agent này nhận context:

$$x = (q, \tilde{K})$$

và candidate set  $\mathcal{O}$ , trong đó  $o^*$  là target item, còn các item khác là negative items. Vì candidate set có thể chứa nhiều item rất giống target, model cần học phân biệt target với các negative mạnh. Do đó paper dùng hard negative sampling:

$$\mathcal{N}_{hard}(x) = \text{Top}N_{o \in \mathcal{O} \setminus \{o^*\}} \pi_\omega(o|x)$$

Tức là lấy các negative mà current policy đánh giá cao nhất.

Với mỗi pair  $(x, o^*, o^-)$ , paper dùng preference optimization loss:

$$\mathcal{L}_{pref} = -\log \sigma \left( \beta \left[ \log \frac{\pi_\omega(o^*|x)}{\pi_{ref}(o^*|x)} - \log \frac{\pi_\omega(o^-|x)}{\pi_{ref}(o^-|x)} \right] \right)$$

Trong đó  $\pi_{ref}$  là reference policy để ổn định training,  $\beta$  là temperature. Loss này khuyến khích policy mới tăng xác suất target item so với hard negatives dưới cùng context.

Để tối ưu toàn bộ framework, paper đề xuất *MMAPO*. MMAPO xem pipeline retrieval–alignment–recommendation như một multi-agent decision process. Retrieval action là chọn expert; alignment actions là sinh các token trong aligned knowledge; recommendation action là chọn item. Vì reward cuối cùng chỉ xuất hiện sau recommendation, MMAPO dùng advantage-based credit assignment.

MMAPO dùng *Generalized Advantage Estimation* (GAE). Với return:

$$G_t = \sum_{l=0}^{T-t} \gamma^l r_{t+l}$$

và temporal-difference residual:

$$\delta_t = r_t + \gamma V(u_{t+1}) - V(u_t)$$

advantage được tính:

$$A_t^{GAE} = \sum_{l=0}^{T-t} (\gamma\lambda)^l \delta_{t+l}$$

Trong đó value baseline  $V(\cdot)$  khác nhau theo step: retrieval agent dùng value network  $V_\phi(s_t)$ , alignment agent dùng value head  $V_\psi(c_t)$  trên generation context.

Reward tổng của MMAPO gồm hai phần:

$$R_{total} = R_{rec} + \alpha R_{MIG}$$

Trong đó recommendation reward:

$$R_{rec} = \begin{cases} \text{conf}(o), & o = o^* \\ -0.1, & \text{otherwise} \end{cases}$$

với:

$$\text{conf}(o) = \pi_\omega(o|q, \tilde{K}, \mathcal{O})$$

là confidence của model đối với item được recommend.

Thành phần mới quan trọng là *Marginal Information Gain* reward:

$$R_{MIG} = \Delta I(e) - \eta C(e)$$

với:

$$\Delta I(e) = D_{KL}(P_{expert} \| P_{base})$$

Trong đó  $P_{expert}$  là recommendation distribution khi dùng expert  $e$ ,  $P_{base}$  là baseline distribution, ví dụ no retrieval, còn  $C(e)$  là computational cost của expert. Expert phức tạp hơn như connected graph có cost cao hơn. Reward này khuyến khích hệ thống chỉ dùng retrieval đắt khi nó tạo ra đủ information gain.

Thực nghiệm được tiến hành trên ba dataset: *MovieLens-1M*, *MovieLens-20M*, và *LFM-1K*. KG được xây dựng từ DBpedia. MovieLens-1M có 6,040 users, 3,883 items và hơn 1 triệu interactions; MovieLens-20M có 138,287 users, 27,278 items và hơn 20 triệu interactions; LFM-1K có 992 users, 107,528 items và hơn 19 triệu interactions. KG tương ứng có số entity và triple khá lớn, ví dụ LFM-1K có 180,457 entities và 516,604 KG triples.

Baselines gồm ba nhóm. Nhóm thứ nhất là zero-shot inference với Gemini-2.0-Flash, GPT-4o, Deepseek-r1, LLaMA3-8B và Mistral-7B. Nhóm thứ hai là LLM-based recommendation models như TallRec và Rec-r1. Nhóm thứ ba là KG-RAG recommendation models như KG-Text, KAPING, G-retriever và K-RagRec. MixRAGRec được đánh giá với hai backbone LLM là LLaMA3-8B và Mistral-7B.

Metric đánh giá gồm:

$$Accuracy, \quad Recall@3, \quad Recall@5$$

Mỗi instance dùng item gần nhất của user làm target, 10 interactions trước đó làm user

context, và candidate pool gồm target item cùng 19 distractor items. Với model chỉ output một option, paper chỉ báo cáo Accuracy.

Kết quả tổng thể cho thấy MixRAGRec vượt tất cả baseline trên cả ba dataset và cả hai backbone. Với LLaMA3-8B, trên MovieLens-1M, MixRAGRec đạt Accuracy 0.504, Recall@3 0.798, Recall@5 0.882, vượt K-RagRec và các LLM-based baselines. Trên MovieLens-20M, MixRAGRec đạt Accuracy 0.676, Recall@3 0.900, Recall@5 0.940. Trên LFM-1K, kết quả rất cao: Accuracy 0.934, Recall@3 0.990, Recall@5 0.992.

Với Mistral-7B, MixRAGRec cũng đạt kết quả tốt nhất. Trên MovieLens-1M, Accuracy đạt 0.518, Recall@3 0.785, Recall@5 0.879. Trên MovieLens-20M, Accuracy đạt 0.652, Recall@3 0.892, Recall@5 0.935. Trên LFM-1K, Accuracy đạt 0.912, Recall@3 0.953, Recall@5 0.962. Điều này cho thấy framework không phụ thuộc vào một backbone duy nhất.

Efficiency study trên MovieLens-1M với LLaMA3-8B cho thấy MixRAGRec vừa chính xác hơn vừa hiệu quả hơn nhiều KG-RAG baselines. K-RagRec đạt Accuracy 0.454 nhưng retrieval time 2.867s và total time 3.776s. MixRAGRec đạt Accuracy 0.504 nhưng retrieval time chỉ 0.063s và total time 1.563s. Điều này cho thấy learned expert selection giúp tránh retrieval nặng không cần thiết.

Khi xét từng expert riêng, retrieval càng sâu thì accuracy càng tăng nhưng cost cũng tăng. Expert 2 TripleRetriever đạt Accuracy 0.488 với retrieval time 0.066s. Expert 3 SubgraphRetriever đạt 0.491 với 0.212s. Expert 4 ConnectedGraphRetriever đạt 0.499 với 0.374s. MixRAGRec đạt 0.504 nhưng retrieval time gần Expert 2, chứng tỏ policy chỉ dùng expensive expert khi cần.

Ablation study xác nhận từng thành phần đều quan trọng. Khi thay MoE selector bằng random expert selection, hiệu năng giảm. Khi bỏ Knowledge Preference Alignment Agent và chỉ dùng templated text, hiệu năng giảm mạnh nhất, đặc biệt trên LFM-1K. Điều này chứng minh alignment là bước then chốt để chuyển KG structure thành text hữu ích cho LLM. Khi bỏ hard negatives hoặc bỏ preference optimization, Accuracy và Recall@K đều giảm, cho thấy contrastive feedback giúp model phân biệt target với competitive negatives.

Phân tích hyperparameter cho thấy MIG weight  $\alpha$  điều khiển trade-off giữa utility và cost. Nếu  $\alpha = 0$ , reward chỉ dựa vào recommendation accuracy, làm retrieval time dài hơn và ranking quality kém hơn. Khi  $\alpha = 0.2$ , Recall@3/5 tốt nhất và retrieval time giảm đáng kể. Nếu  $\alpha$  quá cao như 0.5, model ưu tiên cost quá mạnh làm performance giảm. Vì vậy, cost-aware retrieval cần mức MIG vừa phải.

Số hard negatives  $N$  cũng quan trọng. Nếu  $N = 1$ , contrastive signal quá yếu và performance giảm mạnh. Tăng  $N$  lên 5 hoặc 10 cải thiện Accuracy và Recall@K. Khi tăng đến 15, gain nhỏ hơn và có trade-off giữa các metric, nên paper chọn  $N = 10$ .

Retrieval budget  $M$  kiểm soát số triples, seed entities hoặc retained nodes. Nếu  $M$  quá nhỏ, hệ thống bỏ lỡ tri thức hữu ích; nếu quá lớn, hệ thống đưa thêm nhiễu và tăng

burden cho alignment/recommendation. Trên MovieLens-1M,  $M = 3$  hoặc  $M = 5$  cho kết quả tốt hơn, thể hiện rõ trade-off giữa coverage và noise.

## 2 Conclusion

Hướng đi đã làm của bài báo là xây dựng một KG-RAG recommender có khả năng chọn retrieval granularity theo từng query. Thay vì dùng một chiến lược cố định như luôn lấy triples hoặc luôn lấy subgraph, MixRAGRec dùng Mixture-of-Experts Retrieval Agent để chọn giữa no retrieval, triple retrieval, subgraph retrieval và connected graph retrieval. Cách này giải quyết trực tiếp vấn đề over-retrieval ở query đơn giản và under-retrieval ở query phức tạp.

Đóng góp kỹ thuật quan trọng nhất gồm ba phần. Thứ nhất là *multi-granularity KG retrieval* với bốn expert từ rẻ đến đắt, từ no retrieval đến connected graph. Thứ hai là *Knowledge Preference Alignment Agent*, giúp chuyển graph-structured knowledge thành natural-language snippet có ích cho LLM-based recommendation. Thứ ba là *MMAPO*, một framework policy optimization đa agent dùng shared reward gồm recommendation reward và marginal information gain reward để học expert selection cost-aware.

Kết quả thực nghiệm cho thấy MixRAGRec vượt các baseline zero-shot LLM, LLM-based recommender và KG-RAG recommender trên MovieLens-1M, MovieLens-20M và LFM-1K. Đáng chú ý, MixRAGRec không chỉ tăng Accuracy mà còn cải thiện Recall@3 và Recall@5, tức nâng chất lượng top-K recommendation. Efficiency study cũng cho thấy MixRAGRec đạt accuracy cao hơn K-RagRec nhưng retrieval time và total time thấp hơn nhiều, chứng minh query-aware retrieval có lợi cả về hiệu năng lẫn chi phí.

Ý nghĩa chính của paper là KG-RAG cho recommender system không nên chỉ tập trung vào việc “retrieve more knowledge”. Tri thức nhiều hơn không phải lúc nào cũng tốt hơn, vì có thể gây nhiễu, tăng latency và làm LLM khó tập trung vào thông tin quan trọng. Hướng đúng hơn là retrieve đúng mức, align đúng cách, và tối ưu retrieval policy dựa trên downstream recommendation utility.

Hạn chế tiềm ẩn của hướng này là framework khá phức tạp vì phải huấn luyện và phối hợp nhiều agent: retrieval agent, alignment agent và recommendation agent. Ngoài ra, chất lượng KG, coverage của item trong KG, và khả năng verbalize graph thành text vẫn ảnh hưởng lớn đến kết quả. Với các domain mà KG nghèo, thiếu cập nhật hoặc item không map tốt sang KG entities, hiệu quả của MixRAGRec có thể giảm.

Hướng kế thừa từ bài báo có thể gồm nhiều nhánh. Thứ nhất, có thể mở rộng expert set bằng nhiều dạng retrieval khác như path retrieval, community retrieval, temporal KG retrieval hoặc personalized KG retrieval. Thứ hai, có thể thay alignment agent bằng graph encoder kết hợp với LLM adapter để giữ cấu trúc graph tốt hơn thay vì chỉ chuyển thành text. Thứ ba, có thể áp dụng MixRAGRec cho e-commerce, news, ads hoặc content recommendation, nơi query complexity và knowledge granularity thay đổi mạnh theo từng

user intent.

Ngoài ra, paper này rất phù hợp để kế thừa trong hướng *LLM-based context-aware recommendation*. User query, lịch sử tương tác và KG knowledge có thể được xem là các nguồn context khác nhau. MixRAGRec cho thấy một hướng quan trọng: hệ thống không chỉ cần biết user thích gì, mà còn cần biết query hiện tại cần loại tri thức nào, mức retrieval nào, và cách diễn đạt tri thức đó ra sao để LLM dùng hiệu quả. Trong tương lai, có thể kết hợp MixRAGRec với temporal context, session context hoặc persona context để xây dựng recommender vừa knowledge-grounded, vừa query-aware, vừa cost-aware. ““